

# LAYER 2: STATE & EXECUTION LAYER

## INTEGRATION GATES FOR AGENTIC CODING

### Overview

These Integration Gates define the strict contract boundaries, interfaces, and interaction patterns that any agentic coding system must implement to integrate with **Layer 2 (State & Execution Layer)** of the DAO Governance Architecture.

### Layer 2 Constraints (Non-Negotiable)

| Constraint                                          | Implication for Agents                                                                 |
|-----------------------------------------------------|----------------------------------------------------------------------------------------|
| Non-upgradable core logic                           | Agents cannot patch contracts; all logic must be correct at deployment                 |
| Purely deterministic execution                      | Same inputs must always produce same outputs; no randomness, no external calls in core |
| No external dependencies for core state transitions | Voting, tallying, execution must not depend on oracles, AI, or off-chain data          |

### GATE 1: IDENTITY GATE

**Direction:** Layer 1 (Identity & Sybil-Resistance) → Layer 2

**Purpose:** Validate and ingest verified identity proofs from Layer 1 before any quadratic voting operations can occur.

**Function:** `verifyIdentity()`

**Input Parameters:**

| Parameter                   | Type                 | Description                                                   |
|-----------------------------|----------------------|---------------------------------------------------------------|
| <code>identityProof</code>  | <code>bytes</code>   | ZK-proof generated by Layer 1 identity provider               |
| <code>did</code>            | <code>string</code>  | Decentralized Identifier (e.g., <code>did:ethr:0x...</code> ) |
| <code>credentialHash</code> | <code>bytes32</code> | Hash of soulbound credential token                            |
| <code>timestamp</code>      | <code>uint256</code> | Unix timestamp when proof was generated                       |

**Output:**

| Parameter             | Type    | Description                              |
|-----------------------|---------|------------------------------------------|
| status                | enum    | { VALID, EXPIRED, REVOKED, COMPROMISED } |
| voiceCreditAllocation | uint256 | Credits assigned to this identity        |
| nonce                 | uint256 | Replay attack prevention nonce           |

**Deterministic Validation Rules:**

- **REJECT** if `timestamp > block.timestamp + MAX_PROOF_AGE` (proof too old)
- **REJECT** if `did` exists in `revokedRegistry` (identity previously invalidated)
- **REJECT** if `credentialHash != keccak256(did, issuerPubKey)` (credential mismatch)
- **ACCEPT** → emit `IdentityVerified(did, voiceCreditAllocation, nonce)`

**Function: `invalidateIdentity()`**

**Input Parameters:**

| Parameter       | Type   | Description                              |
|-----------------|--------|------------------------------------------|
| did             | string | Identity to invalidate                   |
| revocationProof | bytes  | Signed revocation from Layer 1 authority |

**Output:**

| Parameter     | Type    | Description                     |
|---------------|---------|---------------------------------|
| success       | bool    | Whether invalidation succeeded  |
| frozenCredits | uint256 | Credits burned or redistributed |

**Deterministic Validation Rules:**

- **REJECT** if caller is Layer 3 (AI) — only Layer 1 or Layer 5 can revoke
- **REJECT** if `did` not in `verifiedRegistry`
- **BURN** all voice credits associated with `did`
- **ADD** `did` to `revokedRegistry` (permanent, immutable append-only)
- Emit `IdentityInvalidated(did, frozenCredits, block.timestamp)`

**Agentic Coding Requirements**

- Agent must verify ZK-proof structure before forwarding to contract
- Agent must maintain local cache of `revokedRegistry` for fast lookups
- Agent must **NEVER** cache identity proofs (privacy requirement)
- Agent must validate proof age against `MAX_PROOF_AGE` constant

## GATE 2: VOICE CREDIT GATE

**Direction:** Internal Layer 2 Ledger

**Purpose:** Manage the immutable ledger of Voice Credits with pure quadratic cost computation. This is the mathematical heart of the system.

**Function:** `getVoiceCredits()`

**Input Parameters:**

| Parameter | Type   | Description       |
|-----------|--------|-------------------|
| did       | string | Identity to query |

**Output:**

| Parameter             | Type    | Description                                 |
|-----------------------|---------|---------------------------------------------|
| availableCredits      | uint256 | Credits available for voting                |
| lockedCredits         | uint256 | Credits locked in active proposals          |
| regenerationRate      | uint256 | Credits regenerated per block               |
| lastRegenerationBlock | uint256 | Last block when regeneration was calculated |

**Deterministic Formula:**

```
availableCredits = min(  
    baseAllocation + (currentBlock - lastRegenerationBlock) * regenerationRate,  
    MAX_CREDIT_CAP  
)
```

- No external oracle dependency for regeneration
- Cap enforced at `MAX_CREDIT_CAP` (hardcoded immutable constant)

**Function:** `calculateQuadraticCost()`

**Input Parameters:**

| Parameter        | Type      | Description                       |
|------------------|-----------|-----------------------------------|
| voteDistribution | uint256[] | Votes per option, e.g., [3, 5, 2] |

**Output:**

| Parameter     | Type      | Description                  |
|---------------|-----------|------------------------------|
| totalCost     | uint256   | Total voice credits consumed |
| perOptionCost | uint256[] | Cost per individual option   |

### Mathematical Formula:

```
totalCost =  $\sum (v_i)^2$  for all i in voteDistribution
```

Example: [3, 5, 2]  $\rightarrow 3^2 + 5^2 + 2^2 = 9 + 25 + 4 = 38$  credits

```
perOptionCost = [9, 25, 4]
```

### Deterministic Rules:

- **Pure function** — no state access, no external calls
- **REVERT** if any  $v_i^2 > \text{MAX\_UINT256}$  (overflow protection)
- Gas-optimized: use unchecked blocks where mathematically safe

### Function: `lockCredits()`

#### Input Parameters:

| Parameter  | Type    | Description     |
|------------|---------|-----------------|
| did        | string  | Voter identity  |
| proposalId | bytes32 | Target proposal |
| amount     | uint256 | Credits to lock |

#### Output:

| Parameter           | Type    | Description               |
|---------------------|---------|---------------------------|
| success             | bool    | Lock success              |
| newAvailableBalance | uint256 | Updated available balance |
| newLockedBalance    | uint256 | Updated locked balance    |

### Deterministic Rules:

- **REJECT** if `amount > availableCredits`
- **REJECT** if `proposalId` not in `ACTIVE_PROPOSAL_REGISTRY`
- **ATOMIC** deduct from available, add to locked
- Emit `CreditsLocked(did, proposalId, amount, block.timestamp)`

### Function: `releaseCredits()`

#### Input Parameters:

| Parameter  | Type    | Description              |
|------------|---------|--------------------------|
| did        | string  | Voter identity           |
| proposalId | bytes32 | Proposal to release from |

#### Output:

| Parameter           | Type    | Description               |
|---------------------|---------|---------------------------|
| releasedAmount      | uint256 | Credits returned          |
| newAvailableBalance | uint256 | Updated available balance |

**Deterministic Rules:**

- **AUTO-TRIGGERED** when proposal state changes to:
- EXECUTED → return credits (proposal passed)
- REJECTED → return credits (proposal failed)
- EXPIRED → return credits (voting timeout)
- **CANNOT** be called manually by user (prevents griefing attacks)
- Emit CreditsReleased(did, proposalId, releasedAmount, block.timestamp)

**Agentic Coding Requirements**

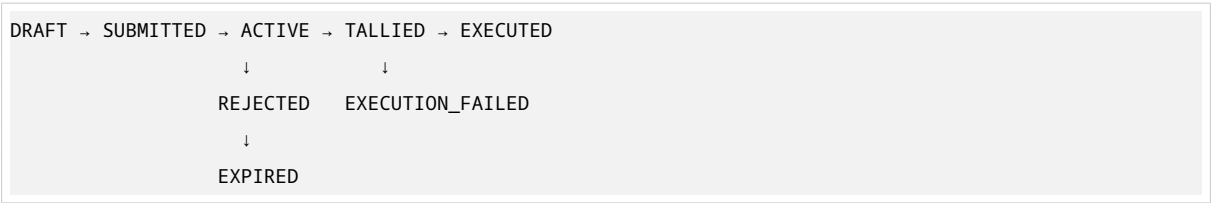
- Agent must pre-compute quadratic cost off-chain for UX optimization
- Agent must validate voteDistribution sums before submitting
- Agent must handle credit locking edge cases (proposal expiry, emergency stop)
- Agent must implement credit regeneration preview for users

**GATE 3: PROPOSAL GATE**

**Direction:** Users → Layer 2

**Purpose:** Handle proposal lifecycle: submission, voting, execution, and state transitions. All state transitions must be purely deterministic.

**Proposal State Machine**



**Function:** submitProposal()

**Input Parameters:**

| Parameter        | Type    | Description                         |
|------------------|---------|-------------------------------------|
| proposerDid      | string  | Identity of proposer                |
| proposalHash     | bytes32 | IPFS hash of full proposal document |
| executionPayload | bytes   | Encoded function call for execution |

Table 14 – continued

| Parameter     | Type      | Description                                        |
|---------------|-----------|----------------------------------------------------|
| votingOptions | bytes32[] | Option labels, e.g., ["FOR", "AGAINST", "ABSTAIN"] |
| metadataURI   | string    | IPFS URI for off-chain metadata                    |

**Output:**

| Parameter        | Type    | Description                        |
|------------------|---------|------------------------------------|
| proposalId       | bytes32 | Unique identifier (keccak256 hash) |
| votingStartBlock | uint256 | Block when voting begins           |
| votingEndBlock   | uint256 | Block when voting ends             |

**Deterministic Rules:**

- **REJECT** if proposerDid not verified via Identity Gate
- **REJECT** if executionPayload calls restricted functions (Layer 2 internals, treasury direct access)
- **REJECT** if votingOptions.length < 2 or > MAX\_OPTIONS
- votingStartBlock = currentBlock + SUBMISSION\_DELAY\_BLOCKS
- votingEndBlock = votingStartBlock + currentVotingDuration
- proposalId = keccak256(abi.encodePacked(allInputs, block.timestamp, block.number))
- Emit ProposalSubmitted(proposalId, proposerDid, votingStartBlock, votingEndBlock)

**Function: castVote()****Input Parameters:**

| Parameter            | Type      | Description                     |
|----------------------|-----------|---------------------------------|
| voterDid             | string    | Identity of voter               |
| proposalId           | bytes32   | Target proposal                 |
| voteDistribution     | uint256[] | Quadratic votes per option      |
| zkProofOfEligibility | bytes     | Optional: for high-stakes votes |

**Output:**

| Parameter        | Type    | Description             |
|------------------|---------|-------------------------|
| voteReceipt      | bytes32 | Unique vote identifier  |
| creditsConsumed  | uint256 | Total credits spent     |
| remainingCredits | uint256 | Credits left after vote |

**Deterministic Rules:**

- **REJECT** if proposal state != ACTIVE
- **REJECT** if currentBlock < votingStartBlock or > votingEndBlock
- **REJECT** if voterDid already voted on this proposal
- **REJECT** if quadraticCost > availableCredits
- Lock credits via Voice Credit Gate
- Store vote as encrypted commitment (commit-reveal pattern for privacy)
- Emit VoteCast(voteReceipt, proposalId, voterDid, creditsConsumed)

**Function: tallyVotes()**

**Input Parameters:**

| Parameter  | Type    | Description       |
|------------|---------|-------------------|
| proposalId | bytes32 | Proposal to tally |

**Output:**

| Parameter          | Type      | Description                          |
|--------------------|-----------|--------------------------------------|
| results            | uint256[] | Total quadratic votes per option     |
| totalParticipation | uint256   | Sum of all sqrt(votes)               |
| quorumMet          | bool      | Whether quorum threshold was reached |
| winningOption      | uint256   | Index of winning option              |

**Deterministic Rules:**

- **REJECT** if currentBlock <= votingEndBlock (voting still active)
- **REJECT** if proposal already tallied
- $results[i] = \sum(\sqrt{voteDistribution_{i,j}})$  for all voters j
- $quorumMet = totalParticipation \geq currentDynamicQuorum$
- $winningOption = \text{argmax}(results)$  — simple plurality
- If tie: winningOption = 0 (first option wins) — deterministic tie-break
- Emit VotesTallied(proposalId, results, quorumMet, winningOption)

**Function: executeProposal()**

**Input Parameters:**

| Parameter  | Type    | Description         |
|------------|---------|---------------------|
| proposalId | bytes32 | Proposal to execute |

Output:

| Parameter       | Type  | Description                        |
|-----------------|-------|------------------------------------|
| success         | bool  | Execution success                  |
| executionResult | bytes | Return data from execution payload |

Deterministic Rules:

- **REJECT** if proposal state != TALLIED
- **REJECT** if !quorumMet
- **REJECT** if winningOption != 0 (assuming 0 = “FOR”/pass)
- **REJECT** if executionPayload calls Layer 2 internal functions
- Execute payload via low-level call (or delegatecall for treasury ops)
- If execution reverts: mark as EXECUTION\_FAILED, return credits to voters
- Emit ProposalExecuted(proposalId, executionResult, block.timestamp)

Agentic Coding Requirements

- Agent must validate executionPayload against restricted function list
- Agent must handle vote commitment/reveal pattern for voter privacy
- Agent must monitor proposal state transitions and auto-trigger tally/execute
- Agent must implement vote simulation before submission
- Agent must track voting windows and send reminders

GATE 4: PARAMETER ADAPTATION GATE

Direction: Layer 3 (Intelligence & Optimization) → Layer 2

**Purpose:** Receive parameter adjustments from Layer 3 (AI Optimization Layer) and apply them within strict, hardcoded bounds. This is the **ONLY mutable surface** of Layer 2.

Adjustable Parameters & Bounds

| Parameter          | Key                | Min Bound      | Max Bound         | Default          |
|--------------------|--------------------|----------------|-------------------|------------------|
| Voting Duration    | VOTING_DURATION    | 100 blocks     | 10,000 blocks     | 1,000 blocks     |
| Quorum Threshold   | QUORUM_THRESHOLD   | 100 credits    | 100,000 credits   | 1,000 credits    |
| Credit Regen Rate  | CREDIT_REGEN_RATE  | 1 credit/block | 100 credits/block | 10 credits/block |
| Max Credit Cap     | MAX_CREDIT_CAP     | 1,000 credits  | 1,000,000 credits | 10,000 credits   |
| Submission Delay   | SUBMISSION_DELAY   | 10 blocks      | 1,000 blocks      | 50 blocks        |
| Proposal Threshold | PROPOSAL_THRESHOLD | 50 credits     | 10,000 credits    | 100 credits      |



**Function: proposeParameterAdjustment()****Input Parameters:**

| Parameter         | Type    | Description                           |
|-------------------|---------|---------------------------------------|
| parameterKey      | enum    | Which parameter to adjust             |
| newValue          | uint256 | Proposed new value                    |
| adjustmentProof   | bytes   | Signed recommendation from Layer 3 AI |
| justificationHash | bytes32 | IPFS hash of AI reasoning document    |

**Output:**

| Parameter      | Type    | Description                  |
|----------------|---------|------------------------------|
| adjustmentId   | bytes32 | Unique adjustment identifier |
| effectiveBlock | uint256 | When adjustment takes effect |

**Deterministic Rules:**

- **REJECT** if `newValue < BOUNDS[parameterKey].min`
- **REJECT** if `newValue > BOUNDS[parameterKey].max`
- **REJECT** if `adjustmentProof` signer not in `AI_ORACLE_WHITELIST`
- **REJECT** if same parameter adjusted `< COOLDOWN_BLOCKS` ago
- `effectiveBlock = currentBlock + TIMELOCK_DURATION_BLOCKS`
- Store in `PENDING_ADJUSTMENTS` with `status = PENDING`
- Emit `ParameterAdjustmentProposed(adjustmentId, parameterKey, newValue, effectiveBlock)`

**Function: applyParameterAdjustment()****Input Parameters:**

| Parameter    | Type    | Description         |
|--------------|---------|---------------------|
| adjustmentId | bytes32 | Adjustment to apply |

**Output:**

| Parameter | Type    | Description              |
|-----------|---------|--------------------------|
| success   | bool    | Application success      |
| oldValue  | uint256 | Previous parameter value |
| newValue  | uint256 | New parameter value      |

**Deterministic Rules:**

- **REJECT** if `currentBlock < effectiveBlock` (timelock not expired)
- **REJECT** if `adjustment status != PENDING`
- **REJECT** if Layer 5 veto flag is set for this `adjustmentId`
- **ATOMIC** update parameter value
- Set `status = APPLIED`
- Emit `ParameterAdjustmentApplied(adjustmentId, oldValue, newValue)`

**Function:** `vetoParameterAdjustment()`

**Input Parameters:**

| Parameter                  | Type                 | Description                     |
|----------------------------|----------------------|---------------------------------|
| <code>adjustmentId</code>  | <code>bytes32</code> | Adjustment to veto              |
| <code>vetoSignature</code> | <code>bytes</code>   | Multisig signature from Layer 5 |

**Output:**

| Parameter            | Type              | Description  |
|----------------------|-------------------|--------------|
| <code>success</code> | <code>bool</code> | Veto success |

**Deterministic Rules:**

- **REJECT** if caller not in `VETO_MULTISIG_SIGNERS`
- **REJECT** if `adjustment status != PENDING`
- Set `status = VETOED`
- Emit `ParameterAdjustmentVetoed(adjustmentId, vetoer)`
- **TRIGGER SAFE MODE:** Revert **ALL** pending adjustments to baseline (Safety Principle)

**Agentic Coding Requirements**

- Agent must validate AI signatures against on-chain whitelist
- Agent must enforce timelock delays strictly (no shortcuts)
- Agent must monitor veto events and trigger safe mode reversion
- Agent must **NEVER** allow Layer 3 to directly modify parameters
- Agent must log all adjustment proposals with full justification

## GATE 5: DATA FEED GATE

**Direction:** Layer 4 (Data & Oracle) → Layer 2

**Purpose:** Receive cryptographically verifiable data from Layer 4 for **read-only analytics** and parameter optimization triggers. Layer 2 **NEVER** depends on this data for core state transitions.

**Function: ingestOracleData()****Input Parameters:**

| Parameter        | Type    | Description                                                                                  |
|------------------|---------|----------------------------------------------------------------------------------------------|
| dataType         | enum    | { VOTER_TURNOUT, TREASURY_VELOCITY, TOKEN_PRICE, PROPOSAL_SUCCESS_RATE, CREDIT_UTILIZATION } |
| value            | uint256 | Reported value                                                                               |
| oracleSignatures | bytes[] | Threshold signatures from Chainlink nodes                                                    |
| roundId          | uint256 | Chainlink round identifier                                                                   |

**Output:**

| Parameter     | Type    | Description                   |
|---------------|---------|-------------------------------|
| verified      | bool    | Signature verification result |
| storedAtBlock | uint256 | Block when data was stored    |

**Deterministic Rules:**

- **NEVER** used in core state transitions (voting, execution, tallying)
- **REJECT** if `oracleSignatures.length < MIN_ORACLE_THRESHOLD`
- **REJECT** if `roundId <= lastProcessedRoundId[dataType]` (stale data)
- Verify threshold signatures (BLS aggregate or ECDSA multisig)
- Store in `analyticsRegistry` (read-only for Layer 3)
- Emit `OracleDataIngested(dataType, value, roundId, block.timestamp)`

**Function: getAnalyticsSnapshot()****Input Parameters:**

| Parameter | Type    | Description         |
|-----------|---------|---------------------|
| dataTypes | enum[]  | Data types to query |
| fromBlock | uint256 | Start block range   |
| toBlock   | uint256 | End block range     |

**Output:**

| Parameter | Type                | Description            |
|-----------|---------------------|------------------------|
| snapshots | AnalyticsSnapshot[] | Historical data points |

**Deterministic Rules:**

- **Pure read function** — no state modification
- Return all ingested data within block range
- Used by Layer 3 for off-chain optimization calculations

**Agentic Coding Requirements**

- Agent must verify multi-source aggregation before ingestion
- Agent must clearly separate “read-only analytics” from “core state”
- Agent must flag any attempt to use oracle data in voting logic
- Agent must implement data freshness checks

---

**GATE 6: EMERGENCY GATE**

**Direction:** Layer 5 (Failsafe / Veto) → Layer 2

**Purpose:** Provide circuit-breaker functionality for Layer 5 to halt operations and revert to safe baseline configuration.

**Function:** `triggerEmergencyStop()`

**Input Parameters:**

| Parameter                       | Type                 | Description                                                                                                                                        |
|---------------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>reason</code>             | <code>enum</code>    | { <code>MALICIOUS_AI</code> , <code>SYBIL_ATTACK</code> , <code>TREASURY_DRAIN</code> , <code>PARAMETER_MANIPULATION</code> , <code>OTHER</code> } |
| <code>evidenceHash</code>       | <code>bytes32</code> | IPFS hash of incident report                                                                                                                       |
| <code>multisigSignatures</code> | <code>bytes[]</code> | Threshold signatures from Layer 5                                                                                                                  |

**Output:**

| Parameter                        | Type                 | Description                    |
|----------------------------------|----------------------|--------------------------------|
| <code>success</code>             | <code>bool</code>    | Stop trigger success           |
| <code>safeModeActivatedAt</code> | <code>uint256</code> | Block when safe mode activated |

**Deterministic Rules:**

- **REJECT** if `multisigSignatures.length < EMERGENCY_THRESHOLD`
- **REJECT** if `safeMode` already active
- Set `safeMode = true`
- **HALT:** proposal submissions, voting, parameter adjustments
- **ALLOW:** emergency withdrawals (if treasury drain suspected)

- **REVERT ALL PARAMETERS** to BASELINE\_CONFIG (immutable constants)
- Emit EmergencyStopTriggered(reason, evidenceHash, block.timestamp)

**Function: liftEmergencyStop()**

**Input Parameters:**

| Parameter          | Type    | Description                        |
|--------------------|---------|------------------------------------|
| resolutionHash     | bytes32 | IPFS hash of governance resolution |
| multisigSignatures | bytes[] | Higher threshold than trigger      |

**Output:**

| Parameter | Type | Description  |
|-----------|------|--------------|
| success   | bool | Lift success |

**Deterministic Rules:**

- **REJECT** if !safeMode
- **REJECT** if multisigSignatures.length < LIFT\_THRESHOLD (must be > trigger threshold)
- **REJECT** if resolutionHash not in approvedResolutions registry
- Set safeMode = false
- Resume normal operations with baseline parameters
- Emit EmergencyStopLifted(resolutionHash, block.timestamp)

**Agentic Coding Requirements**

- Agent must verify higher threshold for lifting than triggering
- Agent must maintain immutable BASELINE\_CONFIG for safe mode reversion
- Agent must log all emergency events with full evidence chain
- Agent must implement automatic notification on emergency trigger

## AGENTIC CODING INTEGRATION PATTERNS

**Pattern 1: Event-Driven Architecture**

Layer 2 Event → Agent Listener → Agent Logic → External Action

**Key Events to Subscribe:**

| Event             | Trigger              | Agent Action                    |
|-------------------|----------------------|---------------------------------|
| IdentityVerified  | New user onboarded   | Initialize user dashboard       |
| ProposalSubmitted | New proposal created | Notify voters, start monitoring |

Table 37 – continued

| Event                       | Trigger            | Agent Action                      |
|-----------------------------|--------------------|-----------------------------------|
| VoteCast                    | Vote recorded      | Update live tally preview         |
| VotesTallied                | Voting ended       | Trigger execution if passed       |
| ParameterAdjustmentProposed | AI suggests change | Start timelock monitoring         |
| ParameterAdjustmentApplied  | Change active      | Update UI, notify users           |
| EmergencyStopTriggered      | Critical incident  | Halt all operations, alert admins |

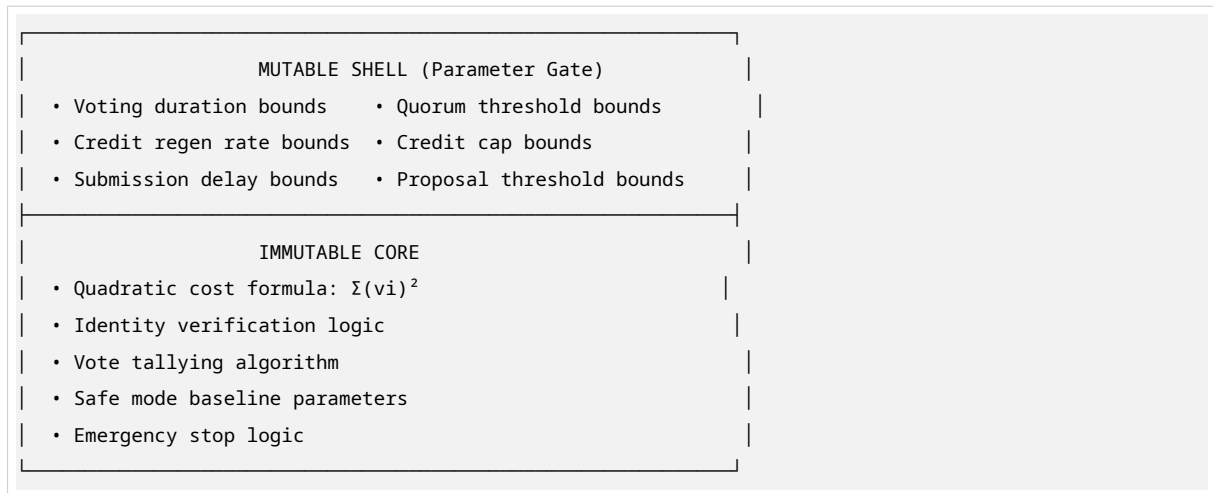
## Pattern 2: Agent-Side Validation Layer

Before any transaction reaches Layer 2, agents must perform:

### Pre-Flight Checklist:

- Simulate transaction locally (`eth_call`)
- Estimate gas and validate user balance
- Check identity verification status
- Validate quadratic cost computation
- Verify proposal state eligibility
- Confirm no pending parameter vetoes affecting this operation
- Check `safeMode` status

## Pattern 3: Immutable Core + Mutable Shell



## Pattern 4: Deterministic Execution Guarantees

Agents must ensure:

- No external calls in vote casting
- No oracle data in proposal execution
- No AI-generated values in core math

- Pure functions for: cost calculation, vote tallying, result determination

## SECURITY CHECKLIST

For each Integration Gate, agents must verify:

| Check                            | Gate 1 | Gate 2 | Gate 3 | Gate 4 | Gate 5 | Gate 6 |
|----------------------------------|--------|--------|--------|--------|--------|--------|
| Reentrancy guards                |        |        |        |        | —      |        |
| Integer overflow protection      | —      |        |        |        | —      | —      |
| Replay attack prevention         |        | —      |        |        | —      |        |
| Access control (layer isolation) |        | —      |        |        |        |        |
| Timelock enforcement             | —      | —      | —      |        | —      | —      |
| Safe mode circuit breaker        | —      |        |        |        | —      |        |
| Audit trail (crypto-graphic)     |        |        |        |        |        |        |
| Gas optimization                 | —      |        |        | —      | —      | —      |

## QUICK REFERENCE: AGENT IMPLEMENTATION PRIORITIES

### Phase 1: Core Voting (Minimum Viable Agent)

1. Implement **Gate 1** (verifyIdentity) — user onboarding
2. Implement **Gate 2** (calculateQuadraticCost, lockCredits) — vote mechanics
3. Implement **Gate 3** (submitProposal, castVote, tallyVotes) — proposal lifecycle

### Phase 2: Optimization (AI Integration)

4. Implement **Gate 4** (proposeParameterAdjustment, applyParameterAdjustment) — self-evolution
5. Implement **Gate 5** (ingestOracleData, getAnalyticsSnapshot) — data feeds

### Phase 3: Safety (Production Hardening)

6. Implement **Gate 6** (triggerEmergencyStop, liftEmergencyStop) — circuit breakers

7. Implement comprehensive event monitoring across all gates
8. Implement agent-side simulation and validation layer

---

*Document generated from DAO Governance Architecture — Layer 2 Integration Gates For agentic coding implementation reference*